

Ruminations on lambda captures

Abstract

The proposal for capturing `*this` by value ([P0018](#)) raised suggestions for a "true value capture", which led to suggestions to change capture-default that defaults to by-value capture(`[=]`) in the case of capturing class members. This paper explores what the suggested changes to the capture-default might mean. This paper specifically doesn't try to claim that any of the changes would have an effect on any particular amount of existing code, and admits that the examples in this paper are somewhat concocted and for illustrative purposes only.

Contents

- [The main suggestions of changed semantics for capturing members](#)
- [Why change anything?](#)
- [The effect of copying the whole object](#)
- [The effect of copying individual members](#)
- [Sure, but aren't such issues already present in lambdas outside classes or class member definitions?](#)
- [Does the proposed `\[\*this\]` suffer from these problems?](#)
- ["Copying members is too hard."](#)
- [""Ok, hotshot, what do you recommend?"](#)

The main suggestions of changed semantics for capturing members

There have been two suggestions for how the semantics of the capture-default for by-value capture of members should change:

1. Capture the whole object by value.
2. Capture the individual members by value.

The first suggestion means roughly the following:

```
struct X
{
    int y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // X::y is used in the lambda, so the whole object
                  // is copied and the X::y of the copy is used here
        };
        // use lam any which way
    }
};
```

The second suggestion means roughly the following:

```
struct X
{
    int y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // X::y is used in the lambda, so X::y
                  // is copied and the copy of that single member is used here
        };
        // use lam any which way
    }
};
```

Why change anything?

The motivation for change is two-fold:

1. A by-value capture-default copies every entity mentioned in the lambda-body, except if the entities are members. This presents a consistency argument between lambdas inside classes and lambdas outside classes or class member definitions.
2. In addition to a mere consistency argument, a by-value capture-default not performing an actual object copy (it will copy the pointer 'this', not the object pointed to nor its members) is suggested to be a pitfall.

The effect of copying the whole object

If the whole surrounding object is copied when a by-value capture-default is used inside a class, we are potentially looking at making currently valid code ill-formed, or changing the semantics of existing code silently.

Here's one example of making currently valid code ill-formed:

```
struct X
{
    unique_ptr<int> y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // *this is attempted to be copied, but that's ill-formed
                  // because X is move-only.
        };
        // use lam any which way
    }
};
```

Here's another example of making currently valid code ill-formed:

```
struct X
{
    mutex y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // *this is attempted to be copied, but that's ill-formed
                  // because X is neither movable nor copyable.
        };
        // use lam any which way
    }
};
```

Here's one example of a silent change of semantics:

```
struct X
{
    vector<int> y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // *this is copied, and X::y with it.
                  // This introduces a copy that wasn't there before,
                  // and means that any code that refers to y will
                  // now refer to the X::y of a copied object, not
                  // the original X::y.
        };
        // use lam any which way
    }
};
```

The extra copy can be a performance issue. The change in which X::y is referred may be a correctness issue.

The effect of copying individual members

As with copying a full object, copying individual members has similar potentials for breakage; it can make currently valid code ill-formed, or change semantics silently.

Here's a slight modification of the move-only example where code becomes ill-formed:

```
struct X
{
    unique_ptr<int> y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // X::y is attempted to be copied, but that's ill-formed
                  // because X::y is move-only.
        };
        // use lam any which way
    }
};
```

In a similar vein, the noncopyable/nonmovable case also breaks:

```
struct X
{
    mutex y;
    void f()
    {
        auto lam = [=]() {
            bar(y); // X::y is attempted to be copied, but that's ill-formed
                  // because X::y is neither movable nor copyable.
        };
        // use lam any which way
    }
};
```

The previous example for a silent change of semantics has the same issues. However, there are different examples:

```
struct X
{
    vector<int> y;
    vector<int>::iterator y_i; // let's assume this iterator points to X::y
    void f()
    {
        auto lam = [=]() {
            bar(y, y_i); // X::y and X::y_i are copied.
                        // This introduces a copy that wasn't there before,
                        // and means that any code that refers to y will
                        // now refer to the X::y of a copied object, not
                        // the original X::y. The copied X::y_i will still
                        // point to the original X::y.
        };
        // use lam any which way
    }
};
```

Yet another example:

```
struct X
{
    vector<int> y;
    vector<int>& y2; // let's assume this reference refers to X::y
    void f()
    {
        auto lam = [=]() {
            bar(y, y2); // X::y and X::y2 are copied.
                      // This introduces two copies that weren't there before,
                      // and means that any code that refers to y will
```

```

        // now refer to the X::y of a copied object, not
        // the original X::y. The copied X::y2 will change
        // from a reference to a vector object.
    };
    // use lam any which way
}
};

```

Sure, but aren't such issues already present in lambdas outside classes or class member definitions?

Sure, turning references into objects and having handles with reference/pointer-semantics refer/point to the original instead of a copy are issues with lambdas that appear outside classes or class member definitions. That's a consistency argument; users need to learn two rules for a by-value capture-default.

The author of this paper thinks the consistency argument is somewhat questionable; outside a class or a class member definition, it's arguably less likely that when capturing multiple entities by value, some of those entities refer to each other in an invariant-preserving way. The author of this paper believes it would be far more likely that class members refer to other members in an invariant-preserving way. Thus making it easier to copy individual members automatically increases the risk of accidentally breaking invariants.

The second counter-argument is compatibility; we may find some amounts of evidence that capture-defaults are rarely used inside classes or class member definitions, or we may find style guides that ban capture-defaults in such contexts; the problem is that there's arguably a lot of code we can't analyze in such ways, and for some of the presented examples, we have user reports according to which there is existing code that relies on the current semantics.

Does the proposed `[*this]` suffer from these problems?

The simple answer is no. It is a pure extension, so it will not break existing code.

"Copying members is too hard."

When the author of this paper explained the potential breakage in the case of move-only or non-copyable/movable types, one response suggested using init-captures. Well, right back at ya, if you want to copy a member, an init-capture will do it; the syntax is explicit, and there's no reliance on "default magic".

Yes, there are cases where init-captures are inconvenient, and P0018 explains some such cases. However, P0018 also provides a solution for most of such cases, which is explicitly copying the whole object, aka `[*this]`. That solution breaks no existing code. There are certain cases which P0018 doesn't cover, but the author of this paper thinks it can be extended to cover those cases if need be, again without breaking any existing code.

"Ok, hotshot, what do you recommend?"

The author of this paper has a fairly straightforward suggestion, and that suggestion requires fairly little work: leave the semantics of by-value capture-defaults unchanged.

The rationale for maintaining the status quo, despite there usually being no requirement to provide any, is three-fold:

1. We won't introduce an incompatibility with C++11 and C++14, we won't make all existing material on lambdas obsolete, and we won't break code, loudly or silently.
2. We can't know how much code we would break. There are reports according to which the amount would be non-zero.
3. To some extent, we won't trade some pitfalls for others.